

Pipeline Improvement Notes

Pipeline Improvement Notes

Purpose

This document is an engineering follow-up note for the current pipeline revision. It is separate from the submission-facing report and is intended to capture the most important technical issues observed in the present build, together with direct, concrete steps to fix them.

The goal is not to redesign the project. The goal is to make targeted changes that improve the final graph while preserving the strengths of the current architecture: automated construction, canonical namespace usage, local-cache reproducibility, provenance capture, and broad source coverage.

Priority 1: Align emitted classes with the generated ontology

Current issue

The current final graph contains high-frequency auxiliary `tfl:` types that are not declared in the generated TBox, including:

- `tfl:TransportOperator`
- `tfl:TransitStop`
- `tfl:Location`
- `tfl:TransportService`
- `tfl:TransportEvent`

This makes the graph harder to query consistently and weakens the relationship between the ontology report and the built artifact.

Direct fix

1. Open `data_retrieval/unstructured_retrieval/triples_to_rdf.py`.
2. Add a canonical class-alignment map before any `rdf:type` assertions are written.
3. Map at least:
 - `TransportOperator` -> `TfLOperator`
 - `TransitStop` -> `TfLStation`
 - `TransportService` -> `TfLLine` or a declared service class, depending on local usage
4. If `Location` and `TransportEvent` are still genuinely useful, declare them explicitly in `generateOntology.py`; otherwise remap or drop them.
5. Add a validation script that fails the build if undeclared `tfl:` classes are emitted into the final graph.

Files to change

- `data_retrieval/unstructured_retrieval/triples_to_rdf.py`

- `generateOntology.py`
- optionally add `scripts/check_ontology_alignment.py`

Priority 2: Restore benchmark-oriented instance coverage for missing competency questions

Current issue

The graph is broad, but several coursework questions still depend on tightly targeted instances that are not yet materialized clearly enough in the final artifact. This is most visible in:

- CQ6 and CQ7 for station facilities
- CQ11 for the peak Oyster fare example
- CQ13 for engineering closures
- CQ15 for the benchmark journey
- CQ20 for fare concessions

Direct fix

1. Create a small post-build enrichment layer containing only benchmark-sensitive instances.
2. Merge that layer after the automated structured and unstructured stages in `main.py`.
3. Keep this layer intentionally small and evidence-backed so it supports the queries without turning the build back into a hand-authored graph.
4. Name benchmark instances consistently, for example:
 - `tfl:PeakFare_Adult_Z1to3`
 - `tfl:Journey_BrixtonToCanaryWharf`
5. Re-run the full SPARQL harness after each added benchmark layer change.

Files to change

- `main.py`
- add `ontologies/manual/benchmark_enrichment.ttl` or a generated equivalent
- optionally add a script that builds benchmark instances from structured inputs

Priority 3: Tighten operator resolution

Current issue

Operator-related queries currently return broad answer sets with multiple plausible organizations instead of one clean benchmark answer.

Direct fix

1. Open `data_retrieval/unstructured_retrieval/triples_to_rdf.py` and inspect the mapping for `operatedBy`.
2. Add an operator canonicalization table so synonyms and historical names resolve to the chosen ontology individual.
3. Restrict `operatedBy` assertions so they attach only to validated line entities.
4. Ensure structured inputs can override noisier text-derived operator assertions when both exist.
5. Re-run CQ8 and CQ14 after each change.

Files to change

- `data_retrieval/unstructured_retrieval/triples_to_rdf.py`

- `data_retrieval/structured_retrieval/tfl_api_processor.py`

Priority 4: Separate Night Tube from night bus evidence more strictly

Current issue

The graph has useful night-service coverage, but the Night Tube query pulls in buses, labels, and non-line entities because the current build treats several night-service subjects too loosely.

Direct fix

1. Add a validation rule that `tfl:isNightTube true` may only be asserted on entities already typed as Underground lines.
2. Keep night buses in `tfl:NightBusRoute` only.
3. In `triples_to_rdf.py`, filter out generic labels such as `Night Tube`, `Night Bus network`, and route-label strings before type assignment.
4. Re-run CQ10 and compare the answer set before and after the filter.

Files to change

- `data_retrieval/unstructured_retrieval/triples_to_rdf.py`
- optionally `data_retrieval/structured_retrieval/tfl_api_processor.py`

Priority 5: Make the live extractor configurable without editing source code

Current issue

The current repository is intentionally cache-backed, which is good for reproducibility, but local reruns are harder than they need to be because `API_KEY`, `model`, and `USE_LLM` are controlled directly in source code.

Direct fix

1. Read `API_KEY`, `model`, and `USE_LLM` from environment variables with sensible defaults.
2. Preserve the current default behaviour so the repo still runs in cache-backed mode out of the box.
3. Document the environment-variable names in the prompt documentation and README.
4. Add a short smoke-test command for local live extraction runs.

Files to change

- `data_retrieval/unstructured_retrieval/extract_triples.py`
- `docs/submission/prompt-documentation.md`
- optionally the repository root `README.md`

Priority 6: Add a post-build validation gate

Current issue

The project already has a competency-question harness, but it would benefit from a second automated gate focused on graph integrity and coursework-sensitive targets.

Direct fix

1. Add a validation script that checks:
 - undeclared `tfl:` classes

- presence of key benchmark instances
 - counts for core classes such as operators, zones, disruptions, and journeys
 - non-empty results for selected high-value SPARQL queries
2. Run this script automatically after graph serialization in `main.py` or as a separate verification command.
 3. Keep the checks simple and deterministic so they are easy to trust.

Files to change

- add `scripts/validate_final_graph.py`
- `main.py`
- optionally `competency_questions/competency_question_test.py`

Suggested Order Of Work

1. Fix ontology alignment for emitted classes.
2. Tighten Night Tube and operator precision.
3. Add benchmark enrichment for fares, journeys, facilities, and disruptions.
4. Add the post-build validation gate.
5. Move live extractor configuration to environment variables.

Expected Payoff

If these steps are implemented, the most likely benefits are:

- cleaner ontology-to-instance alignment
- stronger competency-question precision
- better coverage on missing benchmark queries
- easier local reruns of the unstructured extraction stage
- more confidence that future pipeline changes do not silently reduce graph quality